

MODULE 4.1: SPARSE MODELING, AND DEEP LEARNING

Contents

Sparse Modeling	1
Why Sparse Models?	2
Regularization	2
L ₀ Regularization (Hard Sparsity)	2
L ₁ Regularization (Soft Sparsity / LASSO)	3
L ₂ Regularization (Weight Shrinkage / Ridge)	3
Combined Regularization: Elastic Net	4
Feature Selection through Sparsity	4
Deep Learning.....	5
Motivation for Deep Learning	5
Difference between Machine Learning and Deep Learning	6
Perspectives (Capabilities and Advantages) in Deep Learning	7
Issues (Challenges and Limitations) in Deep Learning	7
Examples of Deep Learning Applications in the Real World.....	7
Perceptron	9
Working of Perceptron Algorithm (Step-by-Step)	9
Neural Network/Artificial Neural Network (ANN)	12
Activation Function	13
Backpropagation Algorithm.....	13
Deep Feedforward Networks (DFNs)/ Multilayer Perceptrons(MLPs)	14

Sparse Modeling

- **Sparse Modeling** is an approach in machine learning and signal processing where we assume that the **underlying data or model parameters contain only a few significant (non-zero) components**, and the rest are zero or negligible. In simple terms:
- Instead of using all features or parameters, we try to **represent data using the smallest possible subset** — this is called *sparsity*.
- Mathematically, if

$$\mathbf{w}=[w_1, w_2, w_3, \dots, w_d]$$

is a parameter vector, and most of the w_i 's are **zero**, then **$\mathbf{w}$ is sparse**.

- In ML, **sparsity** means that **most components of a vector or matrix are zero**. It can apply to:
 - **Model parameters** (e.g., many weights = 0)
 - **Feature vectors** (e.g., only few active words in text data)
 - **Representations** (e.g., images represented by few active basis elements)
- Mathematically: $\text{Sparsity}(\mathbf{w}) = \text{number of zero elements in } \mathbf{w}$

Why Sparse Models?

1. **Reduce Overfitting:** By keeping only important features, sparse models avoid memorizing noise and improve generalization to unseen data.
2. **Improve Interpretability:** Fewer non-zero coefficients make it easier to identify which features truly affect predictions.
3. **Enhance Computational Efficiency:** Sparse models require less storage and faster computation, suitable for large datasets.
4. **Automatic Feature Selection:** Sparsity naturally eliminates irrelevant or redundant features without manual preprocessing.
5. **Noise Reduction:** Ignoring weak or insignificant variables makes predictions more stable.
6. **Compact Representation:** Enables efficient data compression and recovery in applications like signal and image processing.

Regularization

- **Regularization** is a technique used in machine learning to control complexing and **prevent overfitting** in the model by adding a **penalty term** to the loss function.
- It controls model complexity, improves generalization to unseen data, and ensures smoother, more stable predictions by keeping weights small or sparse.
- The general regularized objective function is:

$$J(w) = \text{Loss}(y, \hat{y}) + \lambda \text{Penalty}(w)$$

- $\text{Loss}(y, \hat{y}) \rightarrow$ model's prediction error
- $\lambda \rightarrow$ regularization strength (controls penalty weight)
- $\text{Penalty}(w) \rightarrow$ term depending on the chosen regularization type

L₀ Regularization (Hard Sparsity)

L₀ regularization penalizes the **number of non-zero weights** in the model.

$$C = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m (\omega_j \neq 0)$$

Counts no of non-zero wts in model.
Each non-0 wts adds a penalty of 1

Effect:

- Encourages most weights to become **exactly zero**, creating the **sparsest model**.
- Keeps only the smallest subset of necessary features.

Advantages:

- Produces perfect sparsity — ideal for feature selection.

Disadvantages:

- The function is **non-continuous and non-differentiable**, making optimization **NP-hard**.
- Not used directly in practice — approximated by **L₁**.

Example: If $w = [1.3, 0, 0, -2.1, 0]$, then $\|w\|_0 = 2$ (two non-zero coefficients).

L₁ Regularization (Soft Sparsity / LASSO)

$$\text{Cost Func / Loss Func} = \underbrace{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{Loss Func of a regression model}} + \underbrace{\sum_{i=1}^m |w_i|}_{\text{Penalty}}$$

m = no of features
 n = no of given example
 y_i = actual target value
 $\hat{y}_i$ = predicted target value

Least Absolute Shrinkage and Selection Operator (LASSO) Regression

- A regression model that uses L1 regularization to handle overfitting is called LASSO regression.
- It adds the absolute value of magnitude of the coefficient as a penalty term in the loss function.
- Lasso regression also helps us achieve feature selection by penalizing the weights to approximate equal to zero if that feature does not serve any purpose in the model.

Effect:

- Shrinks some coefficients exactly to **zero**, producing **sparse** solutions.
- Automatically performs **feature selection**.
- Balances simplicity and accuracy.

Advantages:

- Convex and differentiable (piecewise linear)
- Produces interpretable models
- Reduces overfitting by removing irrelevant features

Disadvantages:

- May randomly select one among correlated features instead of keeping all.

Used in:

- **LASSO Regression** (Least Absolute Shrinkage and Selection Operator)
- **Sparse Logistic Regression**

L₂ Regularization (Weight Shrinkage / Ridge)

$$C = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m w_i^2$$

Ridge Regression

- A regression model that uses L2 Regularization.
- This adds the squared magnitude of the coefficient as a penalty term.

Effect:

- Shrinks all weights toward zero but **does not make them exactly zero**.
- Controls model complexity smoothly.

Advantages:

- Reduces variance and overfitting
- Keeps all features (useful when all are informative)
- Stable optimization (differentiable everywhere)

Disadvantages:

- No sparsity → harder to interpret
- Retains all features, even weak ones

Used in:

- **Ridge Regression**
- Neural network weight decay

Combined Regularization: Elastic Net

- **Elastic Net** is a **combined regularization method** that uses both **L₁ (LASSO)** and **L₂ (Ridge)** penalties together in a single model.
- It was introduced to overcome the **limitations of pure LASSO and Ridge regression** by blending their strengths.
- It implies that we add absolute norm as well as the squared measure of the weights.
- To combine L₁'s sparsity and L₂'s stability:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left[(1-\alpha) \sum_{i=1}^m |w_i| + \alpha \sum_{i=1}^m w_i^2 \right]$$

$\alpha \in [0, 1]$ controls the mix:

- $\alpha = 1 \Rightarrow$ LASSO
- $\alpha = 0 \Rightarrow$ Ridge
- $0 < \alpha < 1 \Rightarrow$ Elastic Net

Why Elastic Net is Needed

While L₁ and L₂ are powerful individually, each has drawbacks:

Problem	With LASSO (L ₁)	With Ridge (L ₂)
Highly correlated features	LASSO tends to randomly select <i>one</i> and drop others.	Ridge keeps <i>all</i> but doesn't create sparsity.
Feature selection	Yes, but unstable with correlated variables.	No (shrinks all, none to zero).

✓ Elastic Net solves this by combining both:

- L₁ → induces sparsity (selects important features).
- L₂ → stabilizes coefficients (handles multicollinearity).

Thus, it performs **grouped feature selection** — keeps correlated features together instead of arbitrarily choosing one.

Intuitive View

- **LASSO**: Sparse but unstable when predictors are correlated.
- **Ridge**: Stable but dense (no zero coefficients).
- **Elastic Net**: Balanced — moderately sparse and stable.

Advantages

- Combines feature selection (L₁) and stability (L₂)
- Works well with correlated predictors
- More robust than pure LASSO or Ridge
- Reduces overfitting and variance
- Performs well on high-dimensional datasets ($p > n$)

Feature Selection through Sparsity

- Sparse modeling helps **automatically select features** by enforcing zero weights for irrelevant ones.
- When we apply **L₁ regularization (LASSO)**, the optimization naturally pushes some coefficients to **exactly zero**.
- Features with zero weights are effectively **excluded** from the model.
- Hence, sparsity acts as a built-in **feature selection mechanism**.
- **In short**

- **Feature selection through sparsity** means using regularization (mainly L_1) to keep only the most relevant features by setting the rest of the weights to zero.
- It makes models simpler, faster, and more interpretable without explicit manual selection.
-

Example: Suppose we train a LASSO regression on 10 features and get: $w=[1.5,0,0,-0.8,0,0,2.2,0,0,0.3]$

Only 4 features have non-zero weights → the model automatically selected them as **significant predictors**.

Why It Works

- The penalty term $\lambda \sum |w_i|$ encourages simplicity.
- For small coefficients, it's cheaper (in terms of loss) to set them to zero.
- This yields a model with only the **most informative variables** retained.

Advantages of Sparse Feature Selection

- **Automatic selection:** Model itself decides which features to keep.
- **Improved generalization:** Removes noise and redundancy.
- **Interpretability:** Highlights the most important predictors.
- **Efficiency:** Fewer active features → faster computation.

Deep Learning

- Deep Learning (DL) is a subfield of Machine Learning that uses multi-layered neural networks to automatically learn data representations and complex patterns.
- **Architecture:** It consists of multiple hidden layers between input and output, allowing the network to learn **hierarchical features** — from low-level (edges, words) to high-level (objects, meanings).
- **Learning Method:** It learns by adjusting weights using backpropagation and gradient-based optimization (like SGD or Adam).
- **Key Idea:** Instead of manually designing features, DL learns features automatically from raw data (images, text, sound).
- **Core Models:** Includes CNNs (for images), RNNs and LSTMs (for sequences), Transformers (for NLP), and Autoencoders (for unsupervised representation learning).
- **Data Requirement:** Deep models typically need large datasets and high computational power (GPUs) for effective training.
- **Applications:** Used in vision, speech, NLP, robotics, and autonomous systems.

Motivation for Deep Learning

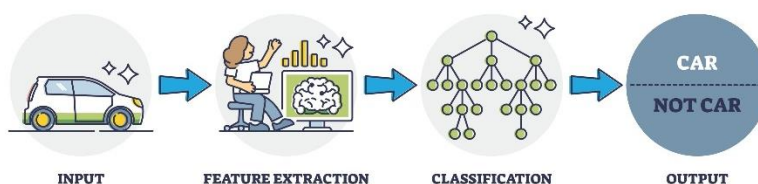
1. **Automatic Feature Extraction:** Traditional ML requires manual feature engineering; DL learns hierarchical features directly from raw input.
2. **Handles High-Dimensional Data:** Works efficiently with images, audio, video, and text — where classical models struggle.
3. **Captures Complex Nonlinear Relationships:** Multiple layers enable modeling of highly nonlinear dependencies in data.
4. **Improved Performance:** Deep networks achieve **state-of-the-art accuracy** in vision, speech, and NLP tasks.

5. **Scalability with Data and Compute:** Performance improves as more data and computational power become available.
6. **Representation Learning:** Learns compact, meaningful representations that generalize well to other tasks (transfer learning).
7. **End-to-End Learning:** Entire process — from raw input to final prediction — can be trained jointly, simplifying system design.

Difference between Machine Learning and Deep Learning

Basis	Machine Learning (ML)	Deep Learning (DL)
Definition	Uses algorithms to learn patterns from data and make predictions.	Subset of ML that uses multi-layer neural networks to learn hierarchical representations.
Feature Engineering	Requires manual feature extraction .	Performs automatic feature learning from raw data.
Model Complexity	Models are relatively shallow (few layers).	Models are deep with multiple hidden layers.
Data Requirement	Works well with small to medium datasets .	Requires large amounts of data for effective training.
Computation Power	Can run on CPUs efficiently.	Needs high-performance GPUs/TPUs for training.
Interpretability	Easier to interpret results and understand decision logic.	Often considered a black box due to complex architectures.
Training Time	Comparatively faster .	Slower — due to many parameters and layers.
Performance	Limited in handling unstructured data.	Excels at processing images, audio, text , and other unstructured data.
Dependency on Domain Knowledge	High — relies on handcrafted features.	Low — automatically discovers useful features.
Examples	Linear Regression, Decision Trees, SVM, k-NN.	CNNs, RNNs, LSTMs, Transformers, Autoencoders.

----- MACHINE LEARNING -----



----- DEEP LEARNING -----



Perspectives (Capabilities and Advantages) in Deep Learning

1. **Automatic Feature Representation Learning:** DL models (especially CNNs and Transformers) **automatically learn complex, hierarchical feature representations** directly from raw data (pixels, text), eliminating the need for manual feature engineering.
2. **State-of-the-Art Performance:** DL holds the **SOTA performance records** across almost every complex task, including image recognition, speech transcription, machine translation, and large language modeling.
3. **Scalability with Data:** DL architectures are inherently **scalable**; their performance generally improves reliably as the volume of training data and the size of the model (parameters) increase.
4. **Handling Unstructured Data:** DL excels at processing **unstructured data** (images, video, text, audio), converting it into meaningful representations (embeddings) that traditional ML models struggle with.
5. **Transfer Learning Efficacy:** The hierarchical feature learning facilitates highly effective **Transfer Learning**, where models pre-trained on massive datasets (like BERT or ResNet) can be fine-tuned for specialized, data-scarce tasks with minimal effort.

Issues (Challenges and Limitations) in Deep Learning

1. **Data Hungry:** DL models are notoriously **data hungry**, requiring vast amounts of labeled data to reach peak performance, which is often expensive and time-consuming to acquire (the labeling bottleneck).
2. **Lack of Interpretability (Black Box):** Deep neural networks are complex "**black box**" models; it's difficult to understand *why* they make a particular prediction, posing significant issues for deployment in critical areas like medicine, finance, and autonomous driving.
3. **High Computational Cost:** Training large, modern models (especially Transformers) requires immense **computational resources** (thousands of GPU/TPU hours), making cutting-edge research and deployment inaccessible without vast infrastructure.
4. **Sensitivity to Data Perturbations:** DL models are highly susceptible to **adversarial attacks**, where tiny, imperceptible changes to the input data (e.g., adding noise to an image) can cause the model to make a confident but catastrophic misclassification.
5. **Brittleness and Generalization:** Models often struggle with **out-of-distribution (OOD) generalization**. They can fail spectacularly when encountering data that differs slightly from the training distribution (e.g., recognizing a familiar object from a novel camera angle).

Examples of Deep Learning Applications in the Real World

❑ Image Recognition (Computer Vision)

- Deep Learning, especially **Convolutional Neural Networks (CNNs)**, has revolutionized image recognition tasks.
- These models automatically learn visual features such as edges, shapes, and objects from raw pixels — without manual feature design. They are used in:
 - **Facial recognition systems** (e.g., phone unlock, surveillance)
 - **Medical imaging** (detecting tumors, fractures, or retinal diseases)
 - **Object detection** in self-driving cars and robotics
 - *Example:* Google Photos automatically classifies and groups similar faces using CNN-based embeddings.

🔊Speech Recognition

- Speech-to-text systems use **Recurrent Neural Networks (RNNs)**, **LSTMs**, or **Transformers** to convert audio signals into text.
- These models learn temporal patterns in sound waves and understand accents, tones, and speech contexts. Used in:
 - Virtual assistants (**Siri, Alexa, Google Assistant**)
 - Automated transcription services
 - Voice-controlled devices and applications
 - *Example:* Google Assistant converts your spoken commands into text using a deep model trained on massive audio datasets.

🗨️Natural Language Processing (NLP)

- Deep learning powers modern NLP systems that understand and generate human language. Models like **BERT**, **GPT**, and **Transformer networks** capture contextual meaning in text using deep attention mechanisms. Applications include:
 - **Chatbots and conversational AI** (customer service)
 - **Language translation** (Google Translate)
 - **Text summarization and sentiment analysis**
 - *Example:* ChatGPT and Google Translate use Transformer-based architectures to generate fluent and context-aware text.

🚗Autonomous Vehicles

- Self-driving cars rely heavily on deep learning for **perception, decision-making, and control**. CNNs and RNNs analyze data from cameras, LiDAR, and radar to identify pedestrians, vehicles, lanes, and traffic signs in real time.
- The system continuously learns to make driving decisions safely and adaptively.
- *Example:* Tesla's Autopilot uses deep neural networks trained on millions of driving scenarios to interpret the road and react automatically.

📺Recommendation Systems

- Deep learning enhances recommendation engines by capturing complex relationships between users and items.
- Neural networks process user history, preferences, and contextual data to predict what the user might like next. Applications include:
 - **Video and music platforms** like Netflix, YouTube, and Spotify
 - **E-commerce** (Amazon product suggestions)
 - *Example:* Netflix uses deep learning to recommend movies based on your viewing patterns and similarity with other users.

🏥Healthcare Diagnostics

- Deep models analyze medical data — such as images, signals, or patient records — to assist in disease diagnosis and prediction.
- CNNs detect **tumors, pneumonia, or diabetic retinopathy** from scans, while RNNs process time-series data like ECGs.
- These models help doctors make faster, more accurate decisions.
- *Example:* DeepMind's eye disease detection system uses retinal images to diagnose conditions as accurately as ophthalmologists.

❑ Finance and Fraud Detection

- Banks and fintech companies use deep networks to detect unusual patterns in transaction data. Recurrent and feedforward neural networks model complex time-based patterns that indicate **fraudulent or suspicious activity**.
- They are also used for **stock price prediction, credit scoring, and risk assessment**.
Example: Mastercard uses deep learning models to detect and block fraudulent transactions in real time.

Perceptron

Perceptron is a supervised learning algorithm used for binary classification problems. It is the simplest type of artificial neural network, where it models a single neuron that learns to classify input data into two classes by adjusting its weights based on the input and the error in prediction.

Objective: To learn a linear boundary (hyperplane) that separates data points of **two classes** using labelled training data.

Perceptron Model: The Perceptron computes a **weighted sum** of the input features and applies an **activation function** (typically the sign function):

$$f(x) = w^T x + b$$

$x = [x_1, x_2, \dots, x_n]$: Input vector

$w = [w_1, w_2, \dots, w_n]$: Weight vector

b : Bias term

$f(x)$: Linear combination of inputs and weights

Output Decision Rule:

$$y = \begin{cases} 1 & \text{if } f(x) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Advantages

1. **Simple to implement:** Easy and intuitive algorithm for binary classification.
2. **Fast for linearly separable data:** Converges quickly when data is separable.
3. **Supports online learning:** Can learn incrementally with new data points.
4. **Forms neural network base:** It's the foundation of complex neural models.
5. **Low computation cost:** Lightweight and efficient for small-scale tasks.

Disadvantages

1. **Can't handle non-linear data:** Fails completely for XOR-like patterns.
2. **Only binary classification:** Needs extension for multi-class problems.
3. **No confidence or probability:** Outputs are hard decisions only.
4. **Sensitive to feature scale:** Needs normalization to avoid bias.
5. **No feature abstraction:** Lacks hidden layers to model complex patterns.

Working of Perceptron Algorithm (Step-by-Step)

1. Initialize Weights and Bias

- Start with small random weights (often 0) and bias $b=0$.
- Set a learning rate η (e.g., 0.01 or 1).

2. Input a Training Sample

- Take an input feature vector $x=[x_1, x_2, \dots, x_n]$
- With its label y (either $+1/-1$ or $0/1$, depending on form used).

3. Compute Output/Weighted Sum

$$z = w^T x + b$$

4. Apply Activation Function

- Use a step function (sign function):

$$\hat{y} = \text{sign}(z) \quad \text{or} \quad \hat{y} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$$

5. Compare Prediction to Actual Label

- If prediction $\hat{y}$ is correct, do nothing.
- If wrong, apply the update rule:

- For label in $\{+1, -1\}$:

$$\begin{aligned} w &= w + \eta y x, \\ b &= b + \eta y \end{aligned}$$

- For label in $\{0, 1\}$:

$$\begin{aligned} w &= w + \eta (y_{\text{true}} - \hat{y}) x, \\ b &= b + \eta (y_{\text{true}} - \hat{y}) \end{aligned}$$

6. Repeat for All Training Samples

- Continue until all data points are classified correctly or max epochs are reached.

Question
 Given: Input $x = [2, 3]$
 Initial wt (w) = $[0, 0]$
 bias (b) = 0
 Learning rate (η) = 1

Calculated updated weight & bias when,
 (a) True label $y = +1$
 (b) True label $y_{\text{true}} = 1$, predicted $\hat{y} = 0$

Answer

(a) $z = w^T x + b = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 2 & 3 \end{bmatrix} = 0$

as $z = 0$, $\hat{y} = \text{sign}(z) = 0$

$\therefore w = w + \eta y x = [0, 0] + 1 \times 1 \times [2, 3] = [2, 3]$

$b = b + \eta y = 0 + 1 \times 1 = 1$

(b) Given $\hat{y} = 0$, (no need to calculate)

$w = w + \eta (y_{\text{true}} - \hat{y}) x = [0, 0] + 1(1-0)[2, 3] = [2, 3]$

$b = w + \eta (y_{\text{true}} - \hat{y}) = 0 + 1(1-0) = 1$

Question (Perceptron Learning Algorithm)

You are given a binary classification problem with the following training data:

Use the **Perceptron Learning Algorithm** to find the final weight vector and bias.

Assume the initial weight vector $w = [0, 0]$, bias $b = 0$, and learning rate $\eta = 1$.

Perform **epoch-wise updates** until convergence.

Use the sign activation function: $\text{sign}(z) = \begin{cases} +1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$

Sample	x1	x2	Class (Label)
1	2	1	+1
2	-1	-3	-1
3	1	-2	-1
4	2	2	+1

Answer

Perceptron Update Rule,

$$\text{If } \hat{y} \neq y, \quad \begin{cases} w = w + \eta y x \\ b = b + \eta y \end{cases}$$

Epoch 1 $w = [0, 0], b = 0$

x	y	
$[2, 1]$	$+1$	$z = w^T x + b = 0$ $\hat{y} = \text{Sign}(z) = +1$ $\therefore \hat{y} = y$ $\therefore$ no update.
$[-1, -3]$	-1	$z = 0$ $\hat{y} = +1$ $\hat{y} \neq y$ Need update. $\therefore w = [0, 0] + 1 \cdot (-1) [-1, -3] = [1, 3]$ $b = 0 + 1 \cdot (-1) = -1$
$[1, -2]$	-1	$z = [1, 3] [1, -2] - 1 = 1 - 6 - 1 = -6$ $\hat{y} = -1$ ($\hat{y} = y$) $\therefore$ no update.
$[2, 2]$	$+1$	$z = [1, 3] [2, 2] - 1 = 2 + 6 - 1 = 7$ $\hat{y} = +1$ ($\hat{y} = y$) $\therefore$ no update.

$$\therefore w = [1, 3] \quad b = -1$$

As 3 samples have converged not all, we have to again repeat the steps.

Epoch 2

x	y	
$[2, 1]$	$+1$	$z = [1, 3] [2, 1] - 1 = 2 + 3 - 1 = 4$ $\hat{y} = +1$ $\therefore$ No update.
$[-1, -3]$	-1	$z = [1, 3] [-1, -3] - 1 = -1 - 9 - 1 = -11$ $\hat{y} = -1$ $\therefore$ No update.
$[1, -2]$	-1	$z = [1, 3] [1, -2] - 1 = 1 - 6 - 1 = -6$ $\hat{y} = -1$ $\therefore$ No update.
$[2, 2]$	$+1$	$z = [1, 3] [2, 2] - 1 = 2 + 6 - 1 = 7$ $\hat{y} = +1$ No update.

$$\therefore \text{At } w = [1, 3] \quad b = -1$$

All samples have converged

This is the decision boundary learned by the Perceptron algorithm,

$$w_1 x_1 + w_2 x_2 + b = 0$$

$$\therefore 1x_1 + 3x_2 - 1 = 0$$

$$\Rightarrow x_1 + 3x_2 = 1$$

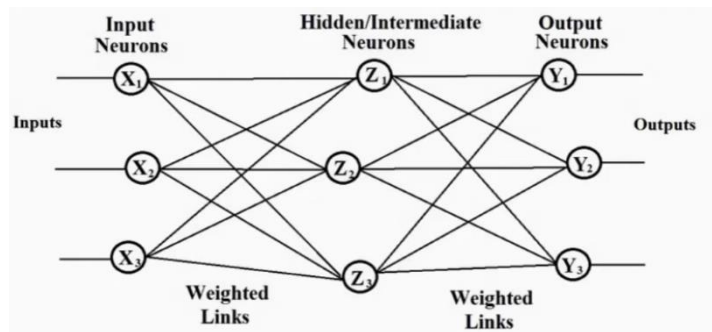
This is a straight line equation, which divides the plane in 2 halves —

→ point above it = classified as $+1$

→ " below = " " -1

Neural Network/Artificial Neural Network (ANN)

An Artificial Neural Network (ANN) is a computational model inspired by the structure and functioning of biological neurons in the brain. It is used to learn patterns from data and make predictions or classifications. It learns from data, adapts, and is useful for both linear and non-linear pattern recognition tasks. **Basic Structure of ANN**



1. Input Layer:

- Takes the raw features (e.g., height, weight).
- Each node represents one feature.

2. Hidden Layer(s):

- Processes inputs with **weights and activation functions**.
- Can be one or many layers deep.
- Learns intermediate representations or features.

3. Output Layer:

- Produces the final result (e.g., class label or regression value).
- Number of nodes = number of output classes (for classification).

How ANN Works (Forward Propagation):

1. Each input x_i is multiplied with a weight w_i .
2. All weighted inputs are summed and added with a bias term b .
3. The result goes through an activation function f :

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$
$$a = f(z)$$

4. This output is passed to the next layer and the process repeats.

Learning in ANN (Training):

- Uses **backpropagation** and **gradient descent** to minimize **error/loss**.
- Weights are updated to reduce the gap between predicted and actual output.

Key Components:

- **Weights (w):** Controls the strength of connections.
- **Bias (b):** Shifts activation, improves flexibility.
- **Activation function (f):** Introduces non-linearity.
- **Loss function:** Measures prediction error.
- **Optimizer (e.g., Gradient Descent):** Updates weights during training.

Applications in Pattern Recognition:

- Handwriting recognition
- Face or object detection
- Speech recognition
- Medical diagnosis (based on symptoms/patterns)
- Fraud detection

Activation Function

An **activation function** is a mathematical function used to **introduce non-linearity** in the output of a computational unit (like a neuron in a model). It is applied to the output of a model's internal computation (typically after calculating a weighted sum of inputs) to determine the final output signal.

Why is it important?

- It **introduces non-linearity** into the model, which helps in **recognizing complex patterns**.
- Without activation functions, a machine learning model (especially neural-like models) would behave like a **linear regression model**, no matter how many layers it has.
- They help the system **decide** whether a neuron (or unit) should be **activated or not**.
- **Control output**: It controls the **range** of the output (like 0 to 1, -1 to 1, etc.).
- **Decision making**: It helps a model decide the class or value based on processed inputs.

Types of Activation records

1. **Step Function**
(Heaviside Function) $f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$
2. **Sign Function** $f(x) = \begin{cases} +1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}$
3. **Sigmoid Function** $f(x) = \frac{1}{1 + e^{-x}}$
Range: (0, 1)
4. **Tanh Function**
(Hyperbolic Tangent) $f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Range: (-1, 1)

Backpropagation Algorithm

Backpropagation (short for *backward propagation of errors*) is the **learning algorithm** used to train multi-layer neural networks by **updating weights efficiently using gradients**.

Steps of Backpropagation

1. **Forward Pass**: Compute output $\hat{y}$ by propagating input through all layers.
2. **Compute Loss**: Measure error between predicted output and actual target using a loss function (e.g., MSE or cross-entropy).

$$L = \frac{1}{2}(y - \hat{y})^2$$

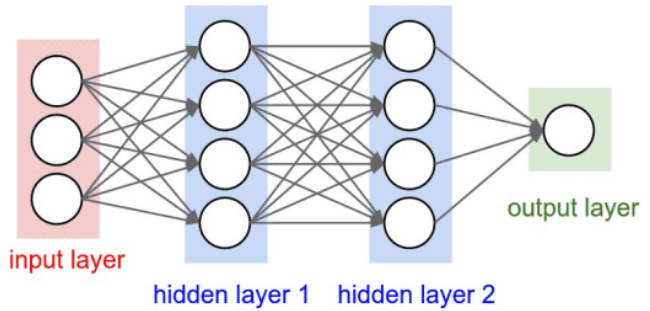
3. **Backward Pass**:
 - Compute **gradients of loss w.r.t. each weight** using the **chain rule**.
 - Propagate errors **backward** from output to input layer.
4. **Weight Update**: Adjust weights to minimize loss:

$$w = w - \eta \frac{\partial L}{\partial w}$$

where η = learning rate.

Deep Feedforward Networks (DFNs)/ Multilayer Perceptrons (MLPs)

- A **Deep Feedforward Network** is a type of **Artificial Neural Network (ANN)** in which information flows **strictly forward** — from the **input layer**, through one or more **hidden layers**, to the **output layer**, without any feedback or loops.
- It's called “feedforward” because data moves forward through the network, and “deep” because it has multiple hidden layers.
- **Direction of Flow:** Forward only (no cycles or feedback loops).
- **Depth:** Defined by number of hidden layers — deeper = more abstract feature learning.
- **Non-linearity:** Introduced via activation functions (ReLU, Sigmoid, etc.).
- **Supervised Learning:** Trained using labeled data and loss functions.
- **Optimization:** Typically trained with stochastic gradient descent (SGD) or its variants.



Structure

Layer	Purpose
Input Layer	Accepts the input features (e.g., pixel values, sensor data).
Hidden Layers	Perform nonlinear transformations and extract higher-level features.
Output Layer	Produces the final prediction or classification result.

Each neuron computes:

$$z = w^T x + b \qquad y = f(z)$$

- x : inputs from previous layer
- w : weight vector
- b : bias
- $f(\cdot)$: activation function (non-linear)

Why Deep?

- A single-layer network (perceptron) can only model **linear relationships**.
- Adding multiple hidden layers allows the network to **approximate complex nonlinear functions** — this is the essence of deep learning.

The *Universal Approximation Theorem* states that a sufficiently large feedforward network can approximate any continuous function.

Advantages

- Can learn **hierarchical feature representations** (from low-level to high-level).
- Handles **nonlinear and complex relationships** in data.
- Foundation for advanced architectures (CNNs, RNNs, Transformers).

Limitations

- Requires **large datasets** and computational power.
- Can **overfit** easily if not regularized (dropout, weight decay, etc.).
- Harder to interpret (black-box behavior).

Applications

- Handwritten digit recognition (e.g., MNIST dataset)
- Credit risk or customer churn prediction
- Stock market forecasting
- Speech and sentiment analysis